



Diplomová práce

Implementace vybraných algoritmů zpracování obrazu v hradlovém poli

Studijní program:

N0613A140028 Informační technologie

Studijní obor:

Výpočetní systémy

Autor práce:

Bc. Karel Najman

Vedoucí práce:

Ing. Martin Rozkovec, Ph.D.

Ústav informačních technologií a elektroniky

Liberec 2026



Zadání diplomové práce

Implementace vybraných algoritmů zpracování obrazu v hradlovém poli

Jméno a příjmení:

Bc. Karel Najman

Osobní číslo:

M24000151

Studijní program:

N0613A140028 Informační technologie

Specializace:

Výpočetní systémy

Zadávací katedra:

Ústav informačních technologií a elektroniky

Akademický rok:

2025/2026

Zásady pro vypracování:

1. Seznamte se s vývojovými nástroji AMD Vitis, Vitis HLS, Vivado a s platformou AMD KV260
2. Proveďte rešerši základních operací počítačového zpracování obrazu a analýzu jejich realizace v HW
3. Vybrané algoritmy implementujte v SW a HW a to pomocí jazyka VHDL a vysokoúrovňové syntézy
4. Výsledky HW implementací porovnejte se SW řešením

<i>Rozsah grafických prací:</i>	Dle potřeby dokumentace
<i>Rozsah pracovní zprávy:</i>	40 – 50 stran
<i>Forma zpracování práce:</i>	tištěná/elektronická
<i>Jazyk práce:</i>	čeština

Seznam odborné literatury:

1. ŠONKA, Milan a HLAVÁČ, Václav. Počítačové vidění. V Praze: Grada, 1992. ISBN 80-85424-67-3
2. HARRIS, David and HARRIS, Sarah, Digital Design and Computer Architecture, Second Edition. V San Francisco. Morgan Kaufmann Publishers Inc., 2012, ISBN 9789382291527.
3. AMD, Inc. Vitis High-Level Synthesis User Guide (UG1399). 2025. Dostupné také z: online <https://docs.amd.com/r/en-US/ug1399-vitis-hls>

<i>Vedoucí práce:</i>	Ing. Martin Rozkovec, Ph.D. Ústav informačních technologií a elektroniky
-----------------------	---

<i>Datum zadání práce:</i>	13. října 2025
<i>Předpokládaný termín odevzdání:</i>	11. května 2026

doc. Ing. Josef Černožorský, Ph.D.
děkan

L.S.

doc. Ing. Petr Červa, Ph.D.
garant studijního programu

Prohlášení

Prohlašuji, že svou diplomovou práci jsem vypracoval samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mé diplomové práce a konzultantem.

Jsem si vědom toho, že na mou diplomovou práci se plně vztahuje zákon č. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.

Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mé diplomové práce pro vnitřní potřebu Technické univerzity v Liberci.

Užiji-li diplomovou práci nebo poskytnu-li licenci k jejímu využití, jsem si vědom povinnosti informovat o této skutečnosti Technickou univerzitou v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.

Současně čestně prohlašuji, že text elektronické podoby práce vložený do IS/STAG se shoduje s textem tištěné podoby práce.

Beru na vědomí, že má diplomová práce bude zveřejněna Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.

Jsem si vědom následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

Implementace vybraných algoritmů zpracování obrazu v hradlovém poli

Abstrakt

Český abstrakt

Klíčová slova: Klíčová slova česky

Implementation of selected image processing algorithms in a gate array

Abstract

English abstract

Keywords: keywords in English

Poděkování

Obsah

1	Úvod	9
2	Nástroje	10
2.1	AMD Vitis™ HLS for Intuitive Design and Productivity	10
2.2	AMD Vivado™ Design Suite	11
2.3	Vývojová deska kria KV260 Vision AI Starter Kit	11
3	HW platforma a metodika návrhu	13
3.1	Platforma Zynq UltraScale+ a Kria KV26	13
3.1.1	LUT	13
3.1.2	FF	13
3.1.3	LUTRAM	13
3.1.4	DSP	13
3.1.5	BRAM	13
4	Reprezentace obrazových dat a barevné prostory	14
4.1	Fyzikální podstata obrazu	14
4.1.1	Viditelné světlo a jeho vnímání	14
4.1.2	Standardní kolorimetrický pozorovatel CIE 1931	14
4.1.3	Akvizice barevného obrazu	16
4.1.4	Barevný model RGB	16
4.1.5	RGB na HSV	18
4.2	Barevný prostor YUV a jeho využití	19
4.2.1	Definice složek Y, U a V	19
4.2.2	Matematické matice převodu z RGB	19
4.2.3	Standard ITU-R BT.2020 (UHDTV)	20
4.2.4	Diagram U-V barevné roviny	20
4.2.5	Chrominanční podvzorkování (Chroma Subsampling)	20
4.2.6	Výhody pro kompresi a zpracování obrazu	21
4.2.7	RGB na YUV	22
4.2.8		22
4.3	Histogram	22
4.4	Filtrace	22
4.4.1	Průměrovací filtr	23
4.4.2	Hranové filtry	23

4.4.3	Morfologické operace	23
5	Hardwarová architektura a implementace algoritmů	25
5.1	Datový tok	25
5.2	Moduly pro barevné transformace	25
5.2.1	Univerzální barevný převodník pro lineární barevné palety?	25
5.2.2	Nelineární barevné převodníky	25
5.2.3	HSV na RGB	25
5.3	Metodologie designu	25
5.3.1	Problém s 2023 a 2025	25
5.4	Aritmetika s pevnou řádovou čárkou	26
5.5	Zpracování video streamu	26
5.6	Převody do barevných soustav	27
5.7	Architektura a hardwarová implementace generického IP jádra pro zpracování obrazu	27
5.7.1	Úvod do architektury IP jádra	27
5.7.2	Rozhraní modulu a datový tok (Dataflow	28
5.7.3	Výpočetní architektura a paralelizace	28
5.7.4	Význam šablonového metaprogramování pro syntézu HW	29
5.7.5	Závěr návrhu	29
5.8	Passthrouht	29
5.9	RGB na HSV	30
5.9.1	VHDL	30
5.9.2	HSL	30
5.10	HSV na RGB	31
5.10.1	VHDL	31
5.10.2	HLS	31
5.11	2D filter	31
5.11.1	Sčítací strom	31
5.11.2	Multiplay-Accumulate MACC	31
5.11.3	HLS	31
5.11.4	VHDL	31
5.11.5	Srovnání	32
6	Závěr	33
	Použitá literatura	34

Seznam zkratek

FPGA	Field Programmable Gate Array	10
IP	Intellectual Property	10
RTL	Register Transfer Level	10
SoC	System on Chip	10
VHDL	VHSIC Hardware Description Language	10

1 Úvod

Motivace, definice problému, cíle práce.

2 Nástroje

2.1 AMD Vitis™ HLS for Intuitive Design and Productivity

AMD Vitis™ HLS je pokročilý vývojový nástroj určený pro automatizovanou syntézu digitálního hardwaru z vysokoúrovňových programovacích jazyků.

Tento nástroj umožňuje transformaci algoritmického popisu v jazycích C/C++ do nízkoúrovňového Register Transfer Level ([RTL](#)) popisu, typicky ve VHSIC Hardware Description Language ([VHDL](#)) nebo Verilogu, optimalizovaného pro Field Programmable Gate Array ([FPGA](#)) a adaptivní System on Chip ([SoC](#)) platformy společnosti AMD.

V kontextu návrhu systémů pro zpracování obrazu představuje Vitis HLS klíčový prvek, který abstrahuje komplexitu hardwarového návrhu a umožňuje soustředit se na funkční stránku algoritmů. Nástroj provádí automatickou analýzu kódu, plánování (scheduling) a mapování operací na dostupné hardwarové zdroje (DSP bloky, paměťové bloky).

Mezi klíčové vlastnosti, využívané v této práci, patří:

- **Hardwarová akcelerace:** Generování vysoce optimalizovaných Intellectual Property ([IP](#)) jader (Intellectual Property cores) s možností definovat výkonnostní parametry, jako je propustnost a latence.
- **Optimalizační direktivy:** Využití tzv. pragmas pro řízení architektury výsledného hardwaru, včetně paralelizace smyček (loop unrolling), pipeliningu instrukcí a správy paměťových rozhraní.
- **Verifikace:** Integrované prostředí pro C-simulaci (ověření funkčnosti algoritmu) a C/RTL ko-simulaci (ověření bitové shody vygenerovaného [RTL](#) s původním C++ modelem).
- **Integrace:** Přímá návaznost na sadu AMD Vivado™ Design Suite pro finální implementaci, syntézu a nahrání bitstreamu do cílového zařízení, např. kitu Kria KV260. a [\[1\]](#)

2.2 AMD Vivado™ Design Suite

AMD Vivado™ je komplexní integrované vývojové prostředí pro SoC a FPGA. Které obsahuje: návrh, syntézu, plánování a mapování, verifikační a simulační nástroje.

Pro účely této práce slouží Vivado jako centrální integrační platforma, která propojuje hardwarové akcelerátory vyvinuté v nástroji Vitis HLS s procesorovým subsystémem (PS) platformy Kria KV260. Klíčovou komponentou prostředí je nástroj IP Integrator, který umožňuje grafický návrh systému na úrovni bloků (Block Design). Zde jsou jednotlivá IP jádra (Intellectual Property cores) propojena pomocí standardizované sběrnice AMBA® AXI, což zajišťuje efektivní komunikaci mezi programovatelnou logikou a řídicím procesorem ARM.

Proces návrhu ve Vivado Design Suite zahrnuje následující klíčové fáze:

- **RTL Syntéza:** Převod zdrojových kódů (VHDL, Verilog) a blokových schémat do seznamu spojů (netlist).
- **Implementace:** Fyzický návrh čipu zahrnující rozmístění (Placement) logických elementů a propojení (Routing) signálových tras. Moderní verze Vivado ML (Machine Learning) využívají v této fázi algoritmy strojového učení pro optimalizaci časování a minimalizaci spotřeby.
- **Verifikace:** Ověření funkčnosti návrhu pomocí behaviorální a post-implementační simulace.
- **Generování bitstreamu:** Vytvoření konfiguračního souboru pro FPGA část a export hardwarové definice (XSA soubor) pro následný vývoj softwaru.

Součástí prostředí je také Vivado Hardware Manager, který umožňuje programování cílového zařízení přes rozhraní JTAG a ladění běžícího hardwaru v reálném čase pomocí integrovaných logických analyzátorů (ILA - Integrated Logic Analyzer).

2.3 Vývojová deska kria KV260 Vision AI Starter Kit

xck26-sfvc784-2lv-c AMD Kria™ KV260 Vision AI Starter Kit je navržena jako výuková platforma pro návrh a vývoj pokročilých aplikací počítačového vidění se zaměřením na akceleraci vizuálních algoritmů a strojového učení. Tento kit je založen na architektuře **Zynq® UltraScale+™ MPSoC** a skládá se z v neprodukčního modulu Kria K26 SOM (System-on-Module) osazeného na nosné desce (carrier card) s bohatou sadou rozhraní pro připojení více kamer a zobrazovacích zařízení pomocí konektoru ON Semiconductor Imager Access System (IAS). Platforma je určena primárně pro vývoj řešení v oblastech, jako jsou chytrá města, průmyslové vidění (machine vision) a bezpečnostní kamery. Díky integrovanému video kodeku (VCU) a programovatelné logice na FPGA umožňuje hardwarovou akceleraci celého aplikačního řetězce – od snímání obrazu přes jeho zpracování až po inferenci neuronových sítí, a to při zachování nízké spotřeby energie a latence. [2]

Tabulka 2.1: Technická specifikace vývojové desky AMD Kria KV260 Vision AI Starter Kit [2]

Komponenta / Rozhraní	Specifikace
SoC (Čip)	Zynq™ UltraScale+™ MPSoC (modul K26 SOM)
Obrazový procesor	OnSemi AP1302 13MP ISP
Programovatelná logika	256 tisíc logických buněk, 144 bloků RAM 64 bloků UltraRAM 1
Paměť DRR	4 GB (4 x 512 Mb x 16 bit)
Kamerová rozhraní	3× MIPI CSI-2 (2× IAS, 1× RPi)
Video výstupy	1× HDMI 1.4, 1× DisplayPort 1.2a
Síťové rozhraní	1× 1Gb Ethernet
Rozšiřující rozhraní	1× PMOD (12-pin)
USB	4× USB 3.0/2.0
Úložiště dat	Primární - 512 Mb QSPI (Queued SPI) Sekundární - slot pro SDF

3 HW platforma a metodika návrhu

3.1 Platforma Zynq UltraScale+ a Kria KV26

3.1.1 LUT

(Look-Up Table): Kombinační logika.

3.1.2 FF

(Flip-Flop / Registry): Paměťové prvky, pipelining.

3.1.3 LUTRAM

(Distributed RAM): Distribuovaná paměť pro malé buffery.

3.1.4 DSP

(Digital Signal Processor): Dedikované slice's (řezy/bloky pro rychlou aritmetiku (MACC operace pro konvoluci a maticové výpočty).

3.1.5 BRAM

Paměťové bloky (Line Buffery, Histogram).

4 Reprezentace obrazových dat a barevné prostory

4.1 Fyzikální podstata obrazu

Proces zpracování obrazových dat úzce souvisí s fyzikální podstatou světla a způsobem, jakým je vnímáno lidským zrakovým ústrojím a následně technicky reprezentováno v digitálních systémech.

4.1.1 Viditelné světlo a jeho vnímání

Světlo je elektromagnetické záření, přičemž lidské oko je citlivé pouze na jeho velmi úzkou část, označovanou jako viditelné spektrum. Toto spektrum zahrnuje vlnové délky přibližně v rozsahu od 380 nm do 750 nm. Různé vlnové délky v tomto intervalu vyvolávají v mozku vjem různých barev – od fialové na spodní hranici (krátké vlnové délky), přes modrou, zelenou, žlutou až po červenou na horní hranici (dlouhé vlnové délky), jak ilustruje obrázek 4.2.

4.1.2 Standardní kolorimetrický pozorovatel CIE 1931

Základem objektivního popisu barev v digitálním zpracování obrazu je model definovaný Mezinárodní komisí pro osvětlování (CIE) v roce 1931. Tento model popisuje průměrné barevné vnímání lidského oka pomocí tří spektrálních funkcí míchání barev: $\bar{x}(\lambda)$, $\bar{y}(\lambda)$ a $\bar{z}(\lambda)$, které odpovídají imaginárním primárním složkám X, Y, Z .

Pro libovolné spektrální složení světla $P(\lambda)$ lze vypočítat odpovídající trichromatické složky X, Y, Z integrací (v praxi sumací přes diskrétní kroky, typicky po 5 nm):

$$X = k \int_{\lambda} P(\lambda) \bar{x}(\lambda) d\lambda, \quad Y = k \int_{\lambda} P(\lambda) \bar{y}(\lambda) d\lambda, \quad Z = k \int_{\lambda} P(\lambda) \bar{z}(\lambda) d\lambda \quad (4.1)$$

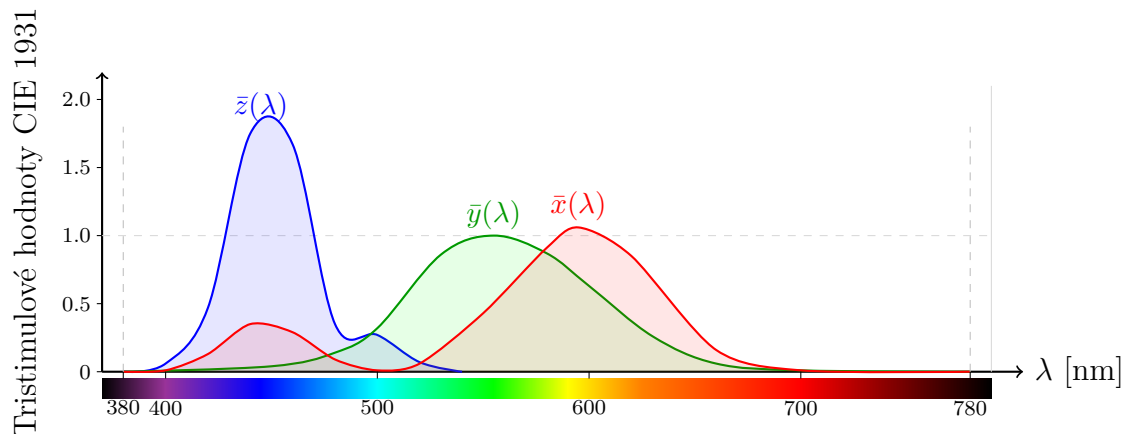
Kde Y přímo reprezentuje jasovou složku (luminanci) a k je normalizační konstanta. Často se využívá normalizovaných trichromatických souřadnic (x, y, z) , které definují pouze chromatičnost (odstín a sytost) nezávisle na jasu, přičemž platí $x + y + z = 1$:

$$x = \frac{X}{X + Y + Z}, \quad y = \frac{Y}{X + Y + Z} \quad (4.2)$$

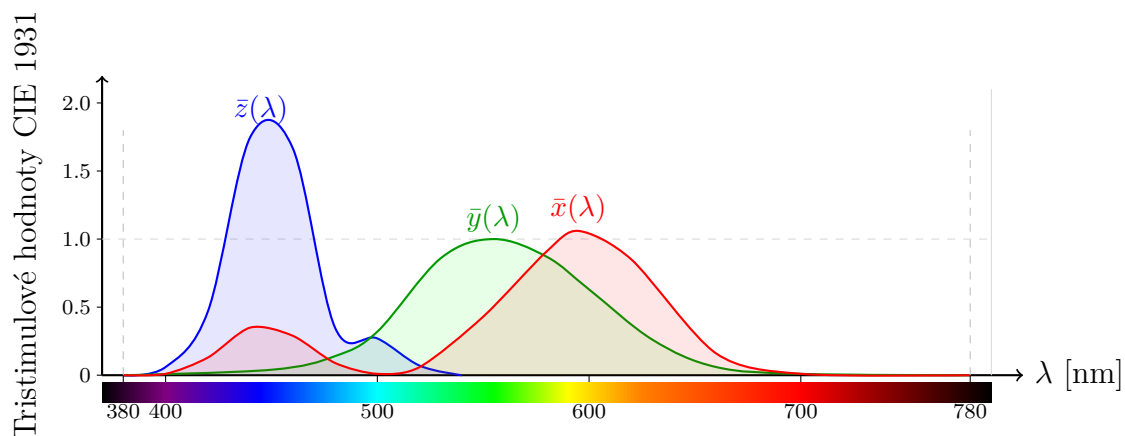
Následná konverze z absolutního prostoru XYZ do konkrétního lineárního RGB prostoru (např. sRGB před gama korekcí) se provádí pomocí standardizované transformační matice $\mathbf{M}$:

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix}_{linear} = \mathbf{M}_{XYZ \rightarrow RGB} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (4.3)$$

Průběh spektrálních funkcí míchání barev CIE 1931 je znázorněn na obrázku 4.1.



Obrázek 4.1: Spektrální funkce míchání barev standardního pozorovatele CIE 1931 (data dle [cie15_2018]). Všimněte si vědecky přesného průběhu funkce $\bar{x}(\lambda)$ (červená), která vykazuje sekundární vrchol v modré oblasti spektra, zatímco funkce $\bar{z}(\lambda)$ (modrá) má vrchol pouze jeden.



Obrázek 4.2: Právě hodnoty trichromatických činitelů standardního pozorovatele CIE 1931 v rozmezí vlnových délek 380–780 nm. Křivka $\bar{y}(\lambda)$ současně reprezentuje poměrnou světelnou účinnost.

Lidské vidění je založeno na trichromatické teorii. Sítnice oka obsahuje světlo-citlivé buňky (čípky) tří typů, které jsou citlivé na krátké (S – modrá), střední (M – zelená) a dlouhé (L – červená) vlnové délky. Z těchto biologických předpokladů přímo vychází technické zpracování obrazu.

4.1.3 Akvizice barevného obrazu

Běžné obrazové senzory v digitálních kamerách (CMOS nebo CCD) dokážou měřit pouze intenzitu dopadajícího světla, tj. počet fotonů. Samy o sobě tedy generují pouze monochromatický (černobílý) obraz a nerozlišují vlnovou délku.

Aby bylo možné zachytit barevný obraz, umísťuje se před jednotlivé pixely senzoru pole barevných filtrů (Color Filter Array – CFA). Nejrozšířenějším typem je tzv. **Bayerova maska**, která propouští ke každému pixelu pouze jednu ze tří základních barev (červenou, zelenou, nebo modrou). Zelených filtrů je na senzoru dvojnásobné množství oproti červeným a modrým, což reflektuje vyšší citlivost lidského oka na zelenou barvu. Ze surových dat senzoru se následně plnobarevný obraz s hodnotami RGB pro každý pixel dopočítává pomocí interpolačních algoritmů, přičemž tento proces se nazývá *demosaiicing*.

4.1.4 Barevný model RGB

Barevný prostor RGB je aditivní barevný model založený na mísení tří základních (primárních) barev: červené (Red), zelené (Green) a modré (Blue). Tento model je absolutním základem digitálního zobrazování a počítačového vidění, neboť přímo odpovídá fyzikální podstatě činnosti obrazovek a výše zmíněných obrazových senzorů.

Princip aditivního míchání

Kombinací tří primárních složek v různých intenzitách vzniká široké spektrum barev. Při nulové intenzitě všech složek nedochází k žádnému vyzařování, čímž získáváme černou barvu. Naopak při jejich maximální a vyvážené intenzitě vzniká barva bílá. Vlnové délky korespondující se základními složkami modelu jsou definovány přibližně následovně:

- **R (Red):** Odpovídá dlouhým vlnovým délkám (~ 630 nm).
- **G (Green):** Odpovídá středním vlnovým délkám (~ 530 nm).
- **B (Blue):** Odpovídá krátkým vlnovým délkám (~ 450 nm).

Technická reprezentace

V počítačové grafice je barva konkrétního obrazového bodu (pixelu) definována uspořádanou trojicí hodnot (R, G, B). Nejčastěji se využívá 8bitová datová hloubka na jeden kanál. Každá složka tedy může nabývat celočíselných hodnot v intervalu $\langle 0, 255 \rangle$. Pro uložení jednoho pixelu v modelu RGB je tak zapotřebí 24 bitů dat

(celkem $2^{24} \approx 16,7$ milionu možných barev). V hardwarových implementacích na hradlových polích (FPGA) je tato bitová šířka stěžejní pro návrh propustnosti paměťových sběrnic (např. AXI4-Stream). Přehled vlastností modelu RGB je shrnut v tabulce 4.1.

Tabulka 4.1: Základní charakteristika barevného modelu RGB

Vlastnost	Popis
Typ modelu	Aditivní (světelný)
Rozsah hodnot	Celočíselně $0 \dots 255$ (8-bit) nebo normalizovaně $0, 0 \dots 1, 0$
Primární využití	Monitory, televizory, digitální kamery, senzory
Hlavní nevýhoda	Vysoká korelace mezi kanály, neseparovaná složka jasu

Standardy a gamuty

Samotný model RGB představuje pouze abstraktní matematický rámec. Pro praktické využití a zajištění barevné konzistence napříč různými zařízeními musí být definován konkrétní barevný prostor (gamut), který přesně určuje chromatické souřadnice primárních barev a referenční bílý bod. Mezi nejužívanější standardy patří:

- **sRGB:** V současnosti nejrozšířenější standard pro webové aplikace a běžná spotřebitelská zařízení.
- **Adobe RGB:** Nabízí výrazně širší barevný gamut, zejména v oblasti zelených a azurových odstínů, což je výhodné pro profesionální zpracování fotografií a předtiskovou přípravu.
- **ProPhoto RGB:** Velmi rozsáhlý barevný prostor používaný primárně při archivaci a špičkovém postprodukčním zpracování obrazu.

Omezení modelu RGB při zpracování obrazu

Ačkoliv je prostor RGB ideální pro zobrazení a akvizici dat, pro složitější počítačové zpracování obrazu nebo kompresi (např. formát JPEG) nebývá optimální. Na rozdíl od prostorů odvozených z modelu YUV (či YCbCr), kde je obraz matematicky rozdělen na jasovou (luminance) a barvovou (chrominance) složku, je v RGB jasová informace úzce provázána a rozprostřena mezi všechny tři kanály. Změna hodnoty jediného kanálu tak ovlivní nejen odstín, ale i celkový jas pixelu. Absence oddělené jasové složky činí formát RGB značně nevhodným pro efektivní datovou kompresi, u které se s výhodou využívá nedokonalosti lidského zraku a jeho nižší citlivosti na změny barev oproti změnám jasu.

4.1.5 RGB na HSV

$$\begin{aligned}
 R' &= \frac{R}{255}, & G' &= \frac{G}{255}, & B' &= \frac{B}{255} \\
 C_{max} &= \max(R', G', B') \\
 C_{min} &= \min(R', G', B') \\
 \Delta &= C_{max} - C_{min}
 \end{aligned}$$

$$H = \begin{cases} 0^\circ & \Delta = 0 \\ 60^\circ \times \left(\frac{G' - B'}{\Delta} \pmod{6} \right) & C_{max} = R' \\ 60^\circ \times \left(\frac{B' - R'}{\Delta} + 2 \right) & C_{max} = G' \\ 60^\circ \times \left(\frac{R' - G'}{\Delta} + 4 \right) & C_{max} = B' \end{cases}$$

$$S = \begin{cases} 0 & C_{max} = 0 \\ \frac{\Delta}{C_{max}} & C_{max} \neq 0 \end{cases}$$

$$V = C_{max}$$

a zpětná transformace

$$\begin{aligned}
 C &= V \times S \\
 X &= C \times (1 - |(H/60^\circ) \pmod{2} - 1|) \\
 m &= V - C
 \end{aligned}$$

$$(R', G', B') = \begin{cases} (C, X, 0) & 0^\circ \leq H < 60^\circ \\ (X, C, 0) & 60^\circ \leq H < 120^\circ \\ (0, C, X) & 120^\circ \leq H < 180^\circ \\ (0, X, C) & 180^\circ \leq H < 240^\circ \\ (X, 0, C) & 240^\circ \leq H < 300^\circ \\ (C, 0, X) & 300^\circ \leq H < 360^\circ \end{cases}$$

$$\begin{aligned}
 R &= (R' + m) \times 255 \\
 G &= (G' + m) \times 255 \\
 B &= (B' + m) \times 255
 \end{aligned}$$

4.2 Barevný prostor YUV a jeho využití

Lidský zrakový systém (HVS – *Human Visual System*) je evolučně uzpůsoben tak, že vykazuje podstatně vyšší citlivost na změny jasu (luminance) než na změny barvy (chrominance). Zatímco barevný prostor RGB je ideální pro zobrazení na koncových zařízeních (monitory, displeje), pro účely zpracování, přenosu a komprese obrazu je krajně neefektivní kvůli vysoké korelaci mezi jednotlivými barevnými kanály. Z tohoto důvodu byl definován barevný prostor YUV, který jasovou a barevnou informaci odděluje.

Obeční výpočetní vzorec:

$$Y = Y_R \cdot R + Y_G \cdot G + Y_B \cdot B$$

$$\text{přičemž platí: } Y_R + Y_G + Y_B = 1$$

$$\begin{aligned} \text{Odvození } P_B \text{ a } P_R: \quad P_B &= 1 - Y_B \\ P_R &= 1 - Y_R \end{aligned} \tag{4.4}$$

$$C_B = \frac{B - Y}{2P_B}$$

$$C_R = \frac{R - Y}{2P_R}$$

4.2.1 Definice složek Y, U a V

Prostor YUV transformuje obraz do jedné jasové a dvou barvonosných (rozdílových) složek. Toto rozdělení vychází z historických potřeb zpětné kompatibility barevného televizního vysílání s černobílými přijímači a je zakotveno ve standardech Mezinárodní telekomunikační unie (ITU).

- **Y (Jasová složka / Luma):** Reprezentuje monochromatickou (černobílou) verzi obrazu. Udává úroveň jasu každého pixelu nezávisle na jeho barvě.
- **U (Chrominance B-Y):** Barevná rozdílová složka reprezentující odchylku modré barvy od celkového jasu.
- **V (Chrominance R-Y):** Barevná rozdílová složka reprezentující odchylku červené barvy od celkového jasu.

Rozdílová složka zelené barvy (G-Y) se nepřenáší, jelikož ji lze matematicky odvodit ze složek Y, U a V, čímž se šetří datový tok.

4.2.2 Matematické matice převodu z RGB

Převod mezi prostory RGB a YUV není fixní, nýbrž závisí na váhových koeficientech definujících příspěvek jednotlivých základních barev do celkového jasu. Tyto koeficienty se vyvíjely v závislosti na použitých luminoforech a moderních zobrazovacích technologiích. Ve všech případech se však vychází z normovaných hodnot $R, G, B \in [0, 1]$.

Standard ITU-R BT.601 (SDTV)

Tento standard byl primárně definován pro televizi se standardním rozlišením (SD-TV). Matice pro dopřednou transformaci z RGB do YUV je definována následovně:

$$\begin{pmatrix} Y \\ U \\ V \end{pmatrix} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ -0.14713 & -0.28886 & 0.436 \\ 0.615 & -0.51499 & -0.10001 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix} \quad (4.5)$$

Standard ITU-R BT.709 (HDTV)

S nástupem televize s vysokým rozlišením (HDTV) byly koeficienty upraveny tak, aby lépe odpovídaly moderním displejům. Zejména se snížil váhový příspěvek červené a modré složky ve prospěch zelené:

$$\begin{pmatrix} Y \\ U \\ V \end{pmatrix} = \begin{pmatrix} 0.2126 & 0.7152 & 0.0722 \\ -0.09991 & -0.33609 & 0.436 \\ 0.615 & -0.55861 & -0.05639 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix} \quad (4.6)$$

4.2.3 Standard ITU-R BT.2020 (UHDTV)

$$\begin{pmatrix} Y \\ U \\ V \end{pmatrix} = \begin{pmatrix} 0.2627 & 0.6780 & 0.0593 \\ -0.1396 & -0.3604 & 0.5000 \\ 0.5000 & -0.4598 & -0.0402 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix} \quad (4.7)$$

Tabulka 4.2: Porovnání primárních převodových koeficientů jasové složky pro standardy ITU-R.

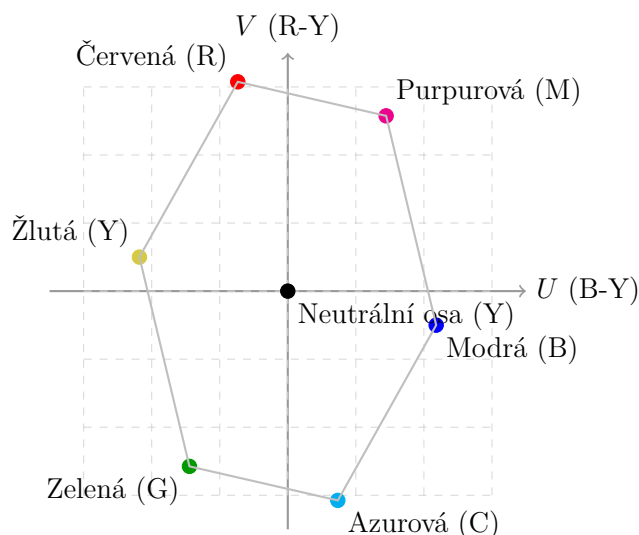
Standard	$\mathbf{W}_R$ (Váha červené)	$\mathbf{W}_G$ (Váha zelené)	$\mathbf{W}_B$ (Váha modré)
ITU-R BT.601 (SD)	0.2990	0.5870	0.1140
ITU-R BT.709 (HD)	0.2126	0.7152	0.0722
ITU-R BT.2020 (4K)	0.2627	0.6780	0.0593

4.2.4 Diagram U-V barevné roviny

Vztah mezi jasem a barvou lze vizualizovat v tzv. U-V rovině (při konstantním jasu Y). Na obrázku 4.3 je zobrazen vektorový prostor chrominančních složek s vyznačením polohy primárních (RGB) a sekundárních (CMY) barev dle standardu BT.601.

4.2.5 Chrominanční podvzorkování (Chroma Subsampling)

Jedním z nejzásadnějších důvodů pro přechod z RGB do YUV v oblasti zpracování obrazu je možnost provedení tzv. *chrominančního podvzorkování* (chroma subsampling). Vzhledem ke zmíněné nižší citlivosti lidského oka na barvu lze prostorové



Obrázek 4.3: Projekce barevného prostoru do 2D chrominanční roviny U-V (tzv. YUV gamut). Neutrální barvy (odstíny šedi) leží v počátku souřadnic ($U=0$, $V=0$).

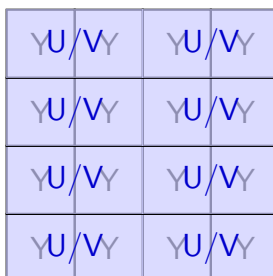
rozlišení složek U a V redukovat, aniž by pozorovatel zaznamenal zjevnou degradaci obrazové kvality.

Systém zápisu podvzorkování se nejčastěji vyjadřuje poměrem $J : a : b$, kde J je šířka referenčního bloku (obvykle 4 pixely), a je počet vzorků chrominance v prvním řádku a b počet vzorků chrominance ve druhém řádku.

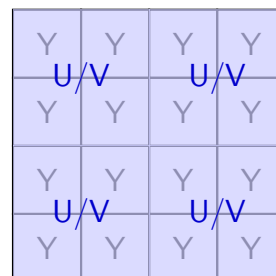
4:4:4 (Bez redukce)



4:2:2 (Redukce 1/2)



4:2:0 (Redukce 3/4)



Obrázek 4.4: Vizualizace principu chrominančního podvzorkování. Každý čtverec v mřížce představuje jeden pixel. Jasová složka (Y) je zachována vždy v plném rozlišení, zatímco barevná informace (U/V) je sdílena pro více sousedících pixelů.

4.2.6 Výhody pro kompresi a zpracování obrazu

Transformace do prostoru YUV poskytuje pro moderní systémy počítačového vidění a kompresní algoritmy (např. JPEG, H.264/AVC, HEVC) následující klíčové benefity:

1. **Datová úspora bez vizuální ztráty:** Aplikací podvzorkování 4:2:0 dochází k redukci celkového objemu dat nekomprimovaného obrazu na polovinu oproti původnímu RGB formátu, a to ještě před aplikací samotných kompresních algoritmů (např. diskrétní kosinové transformace DCT).
2. **Energetická efektivita algoritmů:** Mnoho algoritmů zpracování obrazu (detekce hran, sledování pohybu, extrakce příznaků) nevyžaduje barevnou informaci. Namísto konverze celého RGB obrazu na černobílý stačí v systému YUV pouze vyextrahovat existující kanál Y a provést výpočty nad ním.
3. **Optimalizace kvantizace:** Během kompresního procesu (např. v normě JPEG) jsou používány rozdílné kvantizační matice pro jas a pro barvu. Matice aplikované na chrominanci kanály odstraňují podstatně více vysokofrekvenčních detailů (agresivnější ztrátová komprese) než u kanálu jasového.

dle standardu **SDTV ITU-R BT.601** dle standardu **SDTV ITU-R BT.709**
 FULL HP dle standardu **SDTV ITU-R BT.2020** 4K (možná až 8?)

$$Y = 0.299R + 0.587G + 0.114B$$

$$U = -0.14713R - 0.28886G + 0.436B$$

$$V = 0.615R - 0.51499G - 0.10001B$$

zpětný převod dostaneme inverzní maticí

$$R = Y + 1.13983V$$

$$G = Y - 0.39465U - 0.58060V$$

$$B = Y + 2.03211U$$

4.2.7 RGB na YUV

4.2.8

4.3 Histogram

[Histogram opencv](#)

4.4 Filtrace

Filtrace obrazových dat je jedním s fundamentálních bloků počítačového vidění. Ať už pro vyhlazení hran, odstranění zašumění nebo vylepšení obrazových dat pro další rozpoznávání. Filtrování založená na operaci konvoluce najdeme prakticky všude.

<https://opencv.org/image-filtering-using-convolution-in-opencv/>

$$F(x, y) = (f * g)(x, y) = \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} f(m, n) g(x - m, y - n) \quad (4.8)$$

Ztráta okrajů a zpoždění (Flushing): Pokud chceme režim SAME (výstupní obraz je stejně velký jako vstupní), okno začne generovat platný středový pixel až ve chvíli, kdy do něj nateče celá polovina jádra. Abychom na konci obrazu ze systému "vytlačili" i ty úplně poslední pixely, musí pipeline běžet o něco déle, než je počet pixelů v obraze. Tomuto se říká Flushing.

Identita

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (4.9)$$

4.4.1 Průměrovací filtr

Průměrovací filtr

$$\frac{1}{9} \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (4.10)$$

Zostřovací filtr

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (4.11)$$

4.4.2 Hranové filtry

Prewitt horizontální

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad (4.12)$$

Prewitt vertikální

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad (4.13)$$

Sobel horizontální

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad (4.14)$$

Sobel vertikální

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (4.15)$$

4.4.3 Morfologické operace

V morfologii se tyto matice označují jako strukturní prvky (S).

Čtvercový element (3×3)

$$S_{square} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (4.16)$$

Křížový element (3×3)

$$S_{cross} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad (4.17)$$

Aproximace disku (5×5)

$$S_{disk} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \quad (4.18)$$

5 Hardwarová architektura a implementace algoritmů

5.1 Datový tok

AXI video stream v textu zmínit, že deska podporuje tzv. *overlay* architekturu, což umožní měnit hardwarovou konfiguraci FPGA bez nutnosti restartu celého systému, to je testování algoritmů zpracování obrazu velmi výhodné.

5.2 Moduly pro barevné transformace

5.2.1 Univerzální barevný převodník pro lineární barevné palety?

5.2.2 Nelineární barevné převodníky

RGB na HSV

5.2.3 HSV na RGB

5.3 Metodologie designu

5.3.1 Problém s 2023 a 2025

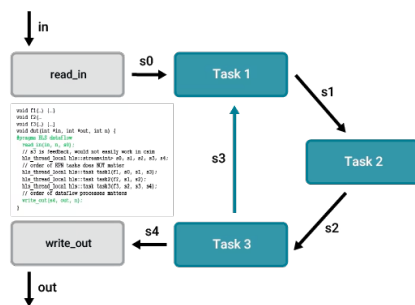
Zásadní problém při implementaci ve verzi 2023.2 je nekompletních konfiguračních souborech (Board Files) pro modul Kria KV260, kterým chybělo detailní fyzické mapování hodinových domén (tzv. Clock Regions) na samotném křemíku Zynq UltraScale $\oplus$. *Kvůli internímu rozvoji algoritmus nedokázal správně alokovat trasu pro asynchronní*

Problém ve verzi 2025.2 je změna flow z nějakého souboru s zatím nevím jakou přípomou se přešlo na xml, jenž to zatím není oddělené a nefuguje jak má.

Programovací model Vitis HLS

Zdrojový kód jazyka C pro Vitis HLS je koncipován tak, aby efektivně využíval výhody a specifické vlastnosti architektury adaptivních SoC a hradlových polí (FPGA) společnosti AMD.

Nástroj Vitis HLS podporuje konstrukty paralelního programování, které umožňují modelovat požadovanou hardwarovou implementaci. Mezi tyto konstrukty patří:



Obrázek 5.1: Enter Caption

UQ7.8	Znaménkový bit	7-bitů pro celé číslo							8-bitů pro desetinné				
Bitová hodnota	-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}

Tabulka 5.1: Caption

- **HLS tasks (úlohy):** umožňují souběžnost na úrovni procesů (process-level concurrency).
- **HLS vectors (vektory):** umožňují paralelismus na úrovni dat (data-level parallelism).
- **HLS streams (proudy):** umožňují komunikaci mezi souběžně běžícími úlohami.

K řízení výsledků syntézy lze využít tzv. **pragmata** (syntetizační direktivy). Tato pragmata zahrnují instrukce pro zřetězení (pipeline), rozvinutí smyček (unroll), dělení paměťových polí (array partitioning) a definici protokolů rozhraní.

Další podrobnosti naleznete v sekci „HLS Programmers Guide“ v uživatelské příručce *Vitis High-Level Synthesis User Guide*.

[1]

5.4 Aritmetika s pevnou řádovou čárkou

formát Q4.4 Texas instrumet nebo ARM [Fixed point](#) článek - Matlab

$$\text{Pro formát } QN.N \text{ Největší číslo } 2^N \text{ Nejmenší číslo } \frac{1}{2^N} \quad (5.1)$$

Příklad převodu $46 * 1 = 46 * 0,5 = 46 * 4,625 =$

5.5 Zpracování video streamu

Neboli AXI stream video signály TLAST a TUSER, tedy EOF (End-of-File) a SOF (Start-of-Frame)

```

void video_proc(input_video , output_video){
for(i=0; i < MAX_FRAME_SIZE; i++){
if(VS){
.....
}
if(HS){
.....
}
if(DE){
//Pixel Processing Algorithm
}
....
}
}
}

```

5.6 Převody do barevných soustav

zmínit problém nalezení toho "správného" postupu YCbCr, YUV, a dalších (

- **Y UV** is the separation used in PAL.
- **YDbDr** is the format used in **SECAM**, unusually based on non-gamma-corrected (linear) RGB, making the Y component true **luminance**.
- **Y IQ** is the format used in **NTSC** television.
- **Y'PbPr** is the separation used in **component video**.
- **Y CbCr** is any **digital encoding** of Y'PbPr suited for video and image compression and transmission formats such as **MPEG** and **JPEG**.^[2]
- **Y CoCg** is a similar separation that offers better performance and lossless round-tripping, used in newer video codecs and **JPEG XR**.

) takže jsem vytvořil univerzální, kde stačí pouze upravit koeficienty

5.7 Architektura a hardwarová implementace generického IP jádra pro zpracování obrazu

5.7.1 Úvod do architektury IP jádra

Navržené IP jádro (Intellectual Property Core) pro konverzi barevných prostorů a bodové operace je koncipováno s důrazem na maximální propustnost, škálovatelnost a efektivní využití hardwarových zdrojů hradlového pole (FPGA). Architektura striktně odděluje fyzické rozhraní modulu (Top-Level) od samotného výpočetního

algoritmu. Tento přístup, využívající pokročilých vlastností jazyka C++ (zejména šablonového metaprogramování – *Template Metaprogramming*), umožňuje generovat vysoce optimalizovaný RTL (Register-Transfer Level) kód na míru konkrétní aplikaci bez nutnosti zásahu do zdrojového kódu algoritmu.

5.7.2 Rozhraní modulu a datový tok (Dataflow)

Procesorový systém a okolní periferie komunikují s IP jádrem pomocí standardizovaných sběrnic rodiny AMBA AXI4.

Datové rozhraní (AXI4-Stream): Pro kontinuální tok obrazových dat je využito rozhraní AXI4-Stream (s_{axis_video} pro vstup, m_{axis_video} pro výstup). Obrazový bod (pixel) je sdružen pomocí direktivy pragma HLS AGGREGATE do jediného datového vodiče (TDATA). Synchronizace snímku je zajištěna postranními signály – začátek snímku (SOF) je indikován signálem TUSER a konec řádku (EOL) signálem TLAST. Díky těmto signálům je jádro navrženo jako datově řízené (*Data-Driven*), což minimalizuje zpoždění a eliminuje nutnost externího stavového automatu (FSM) pro řízení toku.

Řídící rozhraní (AXI4-Lite): Konfigurační parametry, jakými jsou rozlišení obrazu (width, height), transformační matice (coeffs) a aditivní konstanty (offsets), jsou mapovány do paměťového prostoru procesoru prostřednictvím sběrnice AXI4-Lite. To umožňuje běhové (Run-time) modifikace parametrů algoritmu ze strany operačního systému (např. OS Linux běžící na ARM procesoru).

5.7.3 Výpočetní architektura a paralelizace

Hlavní výpočetní smyčka je navržena pro plné zřetězení instrukcí (*Pipelining*). Aplikací direktivy #pragma HLS PIPELINE II=1 je syntezátor Vitis HLS donucen vytvořit hardwarovou strukturu, která je schopna přijmout, zpracovat a odeslat jeden pixel v každém hodinovém taktu (Initiation Interval = 1).

Aby bylo možné dosáhnout propustnosti jednoho pixelu za takt při maticových operacích, bylo nutné překonat omezení čtecích portů standardních blokových pamětí (BRAM).

Destrukce paměťových bloků (Array Partitioning)

Standardní paměť BRAM v FPGA disponuje pouze dvěma čtecími porty. Maticový výpočet však vyžaduje paralelní přístup ke všem koeficientům (např. 9 koeficientů pro matici 3x3) současně. Z tohoto důvodu byla použita direktiva #pragma HLS ARRAY_PARTITION variable=coeffs complete. Tato instrukce rozloží logické pole na samostatné klopné obvody (Flip-Flops), čímž vznikne dedikovaný fyzický vodič pro každý koeficient a je zamezeno vzniku úzkého hrdla při čtení dat.

Využití bloků DSP a ochrana proti přetečení

Vnitřní cyklus, iterující přes barevné kanály, je plně rozbalen pomocí direktivy #pragma HLS UNROLL. Tím je zamezeno vzniku sekvenčního stavového automatu

a namísto toho je vytvořen paralelní strom násobiček.

Syntezátor pro tyto operace (Multiply-Accumulate - MAC) automaticky infere dedikované DSP řezy (např. DSP48E2 na architektuře UltraScale). Pro zajištění numerické stability je akumulací registr definován pomocí datového typu $ap_{int} < 48 >$. Šířka 48 bitů přesně mapuje fyzickou architekturu sčítačky uvnitř bloku DSP, čímž je zaručena naprostá odolnost proti přetečení (*Overflow*) i při kaskádním sčítání dílčích součinů. Následná normalizace výpočtu je provedena bitovým posunem vpravo ($\gg 8$), což v hardwaru nepředstavuje žádnou logickou spotřebu, jedná se pouze o přeznačení vodičů.

5.7.4 Význam šablonového metaprogramování pro syntézu HW

Inovativním prvkem navrženého řešení je definice algoritmu ve formě C šablony ($template < intIN_CH, intIN_W, intOUT_CH, intOUT_W >$). Tento přístup přináší zásadní výhody pro finální hardwarovou implementaci:

1. **Konstantní propagace v době kompilace:** Parametry šablony (počet kanálů, bitová šířka) jsou vyhodnoceny během kompilace (*Compile-time*). Konstanty pro saturaci obrazu (např. MAX_{VAL}) jsou syntezátorem zadrátovány přímo do komparátorů, čímž odpadá nutnost alokace paměti pro tyto mezí hodnoty.
2. **Eliminace mrtvého kódu (Dead-Code Elimination):** Pokud je jádro instanciováno pro konverzi 3kanálového formátu na 1kanálový (např. RGB do Grayscale), HLS nástroj automaticky optimalizuje rozbalené smyčky a fyzicky syntetizuje pouze ty DSP bloky a registry, které jsou pro danou dimenzi matice nezbytné. Neaktivní kanály jsou nástrojem odříznuty, což vede k dramatické úspoře hradlové logiky (LUT) a spotřeby energie na čipu.
3. **Univerzalita modulu:** Jeden zdrojový kód pokrývá libovolné konverze (RGB $\rightarrow$ YCbCr, RGBA $\rightarrow$ CMYK, atd.) a podporuje asymetrické bitové šířky vstupu a výstupu, což odpovídá moderním požadavkům metodiky návrhu znovupoužitelných IP jader (IP Reuse).

5.7.5 Závěr návrhu

Kombinací paralelizace na úrovni dat (Data-Level Parallelism), efektivního mapování DSP bloků a generické metaprogramovací architektury bylo vytvořeno robustní IP jádro. Modul bez problémů splňuje požadavky na zpracování video toku v reálném čase (Real-time Video Processing) s minimální latencí, deterministickým časováním a optimální spotřebou zdrojů platformy FPGA.

5.8 Passthrouht

Testovací komponenta, logicky je nejjednodušší ale byla důležitá k testování a naučení se ovládání datového streamu, důležitý poznatek je dodržet to co Xilinx uvádí ve

své dokumentaci a to je zpracování ve dvou vnořených cyklech, pro výšku a šířku myslím že to je v souboru XAP793 nebo jak se to jmenuje

5.9 RGB na HSV

5.9.1 VHDL

Prostředek	Syntéza	Implementace	Procent (%)
LUT	220	218	0,19
LUTRAM	14	14	0,02
FF	118	118	0,05
DSP	2	2	0,16
IO	58	58	30,69

Tabulka 5.2: Srovnání prostředků

5.9.2 HSL

5.10 HSV na RGB

5.10.1 VHDL

Prostředek	Syntéza	Implementace	Procent (%)
LUT	385	381	0,33
LUTRAM	6	6	0,01
FF	181	181	0,08
DSP	2	2	0,16
IO	58	58	31,69

Tabulka 5.3: Srovnání prostředků

5.10.2 HLS

5.11 2D filter

Filter Designs

5.11.1 Sčítací strom

Adder Tree

5.11.2 Multiplay-Accumulate MACC

<https://docs.amd.com/r/en-US/am004-versal-dsp-engine/MACC-Extension> Vynásob a sečti

5.11.3 HLS

5.11.4 VHDL

Prostředek	Syntéza	Implementace	Procent (%+ – 0.1)
LUT	11368	11355	9,71
LUTRAM	123	123	0,21
FF	33161	33161	14,16
DSP	0	0	0,0
IO	98	98	51,85

Tabulka 5.4: Srovnání prostředků

5.11.5 Srovnání

6 Závěr

Použitá literatura

- [1] AMD. *AMD VitisTM HLS*. 2026. Dostupné také z: <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-hls.html>.
- [2] AMD. *Kria KV260 Vision AI Starter Kit*. 2026. Dostupné také z: <https://www.amd.com/en/products/system-on-modules/kria/k26/kv260-vision-starter-kit.html>.